

June 29, 2026

1 Valgrind e Simulação de Cache: Cachegrind

Este laboratório apresenta o uso da ferramenta cachegrind do ambiente Valgrind, [para maiores informações consulte aqui](#)

Importante: * A primeira execução do Cachegrind irá fazer a instalação da ferramenta e pode demorar um pouco mais. * Os laboratorios usam uma multiplicação de matrizes como exemplo. O tamanho da matriz cresce com $O(N^2)$ e o tempo de execução com $O(N^3)$. * Os exemplos estão em C. Mas o Cachegrind trabalha sobre o executável e pode ser usado em qualquer binário. * Fique a vontade para contribuir.

1.1 Inicialização

Primeiro, configurar o laboratório.

```
[ ]: !pip install git+https://github.com/lesc-ufv/cad4u >& /dev/null
[ ]: !git clone https://github.com/lesc-ufv/cad4u >& /dev/null
[ ]: %load_ext plugin
```

```
[ ]: %%datacache
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char const *argv[]) {

    int n = 100;
    char a[n][n], b[n][n], c[n][n];

    int s = 0;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            a[i][j] = i + j;
            b[i][j] = i*2 + j;
        }
    }

    int temp;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
```

```

        temp = 0;
        for (int k = 0; k < n; ++k) {
            temp += a[i][k] * b[k][j];
        }
        c[i][j] = temp;
    }
}

return 0;
}

```

Installing. Please wait... done!

```

GridspecLayout(children=(Button(button_style='warning', description='Data_
Cache', layout=Layout(grid_area='wid...

```

Parameters: 4096, 2, 32

LLi miss rate: 0.00%

D refs:	12,318,259	(12,256,357 rd	+ 61,902 wr)
D1 misses:	551,326	(542,865 rd	+ 8,461 wr)
LLd misses:	2,099	(1,183 rd	+ 916 wr)
D1 miss rate:	4.5%	(4.4%	+ 13.7%)
LLd miss rate:	0.0%	(0.0%	+ 1.5%)

LL refs:	552,248	(543,787 rd	+ 8,461 wr)
LL misses:	3,011	(2,095 rd	+ 916 wr)
LL miss rate:	0.0%	(0.0%	+ 1.5%)

2 Specify all cache parameters

A extensão `%%cachegrind` é semelhante a linha de comando, importante que os tamanhos de cache devem ser potência de 2, a linha além de potência de 2 começa com 32 bytes. A ordem dos parametros é tamanho da cache, associatividade e tamanho da linha. Os flags para cache de dados, de instruções e de último nível são **D1**, **I1**, and **LL**, respectivamente.

```

[ ]: %%cachegrind --D1=32768,8,32 --I1=32768,2,32 --LL=65536,2,32 --file
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char const *argv[]) {

    int n = 100;
    int a[n][n], b[n][n], c[n][n];

```

```

for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
        a[i][j] = i + j;
        b[i][j] = i*2 + j;
    }
}

int temp;
for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
        temp = 0;
        for (int k = 0; k < n; ++k) {
            temp += a[i][k] * b[k][j];
        }
        c[i][j] = temp;
    }
}
return 0;
}

```

LLi miss rate: 0.01%

D refs:	12,318,285	(12,256,384 rd	+ 61,901 wr)
D1 misses:	133,115	(128,348 rd	+ 4,767 wr)
LLd misses:	9,077	(4,321 rd	+ 4,756 wr)
D1 miss rate:	1.1%	(1.0%	+ 7.7%)
LLd miss rate:	0.1%	(0.0%	+ 7.7%)

LL refs:	134,753	(129,986 rd	+ 4,767 wr)
LL misses:	10,688	(5,932 rd	+ 4,756 wr)
LL miss rate:	0.0%	(0.0%	+ 7.7%)

2.1 Atenção aos resultados

Valgrind quando simula a cache ele também simula a inicialização do sistema. Portanto, quando for utilizar o valgrind esteja ciente que se o seu código for muito simples será mascarado pela inicialização do sistema.

[]: %%cachegrind --D1=1024,8,32 --I1=32768,2,32 --LL=65536,2,32 --file

```

int main(int argc, char const *argv[]) {
    ///  
empty code
}

```

LLi miss rate: 1.02%

D	refs:	46,650	(35,077 rd	+ 11,573 wr)
D1	misses:	13,867	(11,365 rd	+ 2,502 wr)
LLd	misses:	3,240	(2,216 rd	+ 1,024 wr)
D1	miss rate:	29.7%	(32.4%	+ 21.6%)
LLd	miss rate:	6.9%	(6.3%	+ 8.8%)

LL	refs:	15,438	(12,936 rd	+ 2,502 wr)
LL	misses:	4,788	(3,764 rd	+ 1,024 wr)
LL	miss rate:	2.4%	(2.0%	+ 8.8%)

2.2 Exemplo de código mascarado

Abaixo é apresentado um código de **transposição de matrizes** sendo mascarado pela inicialização do sistema.

```
[ ]: %%cachegrind --D1=1024,8,32 --I1=32768,2,32 --LL=65536,2,32 --file
#include <stdio.h>
#include <stdlib.h>

#define n 32
int A[n][n], B[n][n];

void trans(int M, int N) {
    int i, j, tmp;
    for (i = 0; i < N; i++)
        for (j = 0; j < M; j++) {
            tmp = A[i][j];
            B[j][i] = tmp;
        }
}

int main(int argc, char const *argv[]) {
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            A[i][j] = i + j;

    trans(n, n); /// transposição de matrizes
    return 0;
}
```

LLi miss rate: 0.80%

D	refs:	65,413	(50,698 rd	+ 14,715 wr)
D1	misses:	15,154	(11,498 rd	+ 3,656 wr)
LLd	misses:	3,502	(2,222 rd	+ 1,280 wr)
D1	miss rate:	23.2%	(22.7%	+ 24.8%)
LLd	miss rate:	5.4%	(4.4%	+ 8.7%)

LL refs:	16,733	(13,077 rd	+ 3,656 wr)
LL misses:	5,058	(3,778 rd	+ 1,280 wr)
LL miss rate:	1.9%	(1.5%	+ 8.7%)

2.3 Resultados somente inicialização X transposição de matrizes

Somente Inicialização: * D refs: 1,203,059 (771,626 rd + 431,433 wr) * D1 misses: 284,860 (219,936 rd + 64,924 wr)

Transposição de matrizes: * D refs: 1,221,822 (787,247 rd + 434,575 wr) * D1 misses: 286,147 (220,069 rd + 66,078 wr)

Note que a diferença é pequena, sendo para a cache dados L1: * D refs: 1221822 - 1203059 = 18763
* D1 misses: 286147 - 284860 = 1287

2.4 Solução: Mais trabalho para o algoritmo

Uma solução é fazer com que o seu código der mais trabalho para cache de dados, assim a inicialização não irá mascarar os resultados.

2.4.1 Transposição simples

```
[ ]: %%cachegrind --D1=1024,8,32 --I1=32768,2,32 --LL=65536,2,32 --file
#include <stdio.h>
#include <stdlib.h>

#define n 32
int A[n][n], B[n][n];

void trans(int M, int N) {
    int i, j, tmp;
    for (i = 0; i < N; i++)
        for (j = 0; j < M; j++) {
            tmp = A[i][j];
            B[j][i] = tmp;
        }
}

int main(int argc, char const *argv[]) {
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            A[i][j] = i + j;

    for (int i; i < 2000; ++i)
        trans(n, n); //# transposição de matrizes
    return 0;
}
```

LLi miss rate: 0.00%

D refs: 22,987,949 (18,805,319 rd + 4,182,630 wr)
D1 misses: 2,320,001 (269,370 rd + 2,050,631 wr)
LLd misses: 3,502 (2,222 rd + 1,280 wr)
D1 miss rate: 10.1% (1.4% + 49.0%)
LLd miss rate: 0.0% (0.0% + 0.0%)

LL refs: 2,321,581 (270,950 rd + 2,050,631 wr)
LL misses: 5,059 (3,779 rd + 1,280 wr)
LL miss rate: 0.0% (0.0% + 0.0%)

```
[ ]: %%cachegrind --D1=1024,8,32 --I1=32768,2,32 --LL=65536,2,32 --file
#include <stdio.h>
#include <stdlib.h>

#define n 32
int A[n][n], B[n][n];

void transpose_32_32(int M, int N) {
    int BLOCK_SIZE, rowIndex, colIndex, blockedRowIndex, blockedColIndex,
    eBlockDiag1, iBlockDiag1;
    BLOCK_SIZE = 8;
    for (colIndex = 0; colIndex < M; colIndex += BLOCK_SIZE) {
        for (rowIndex = 0; rowIndex < N; rowIndex += BLOCK_SIZE) {
            for (blockedRowIndex = rowIndex; blockedRowIndex < rowIndex +
            BLOCK_SIZE; ++blockedRowIndex) {
                for (blockedColIndex = colIndex; blockedColIndex < colIndex +
            BLOCK_SIZE; ++blockedColIndex) {
                    if (blockedRowIndex != blockedColIndex)
                        B[blockedColIndex][blockedRowIndex] =
            A[blockedRowIndex][blockedColIndex];
                    else {
                        eBlockDiag1 = A[blockedRowIndex][blockedColIndex];
                        iBlockDiag1 = blockedRowIndex;
                    }
                }
                if (colIndex == rowIndex)
                    B[iBlockDiag1][iBlockDiag1] = eBlockDiag1;
            }
        }
    }
}

int main(int argc, char const *argv[]) {
    for (int i = 0; i < n; ++i)
```

```

    for (int j = 0; j < n; ++j)
        A[i][j] = i + j;
    for (int i; i < 2000; ++i)
        transpose_32_32(n, n); //# transposição de matrizes 32x32
    return 0;
}

```

```

I1 misses:          3,581
LLi misses:         3,514
I1 miss rate:       0.01%
LLi miss rate:      0.01%

```

```

D  refs:           29,162,328 (26,245,989 rd  + 2,916,339 wr)
D1 misses:         1,844,982 (  480,575 rd  + 1,364,407 wr)
LLd misses:         48,387  (   21,298 rd  +   27,089 wr)
D1 miss rate:       6.3% (    1.8%   +   46.8%  )
LLd miss rate:      0.2% (    0.1%   +    0.9%  )

```

```

LL refs:           1,848,563 (  484,156 rd  + 1,364,407 wr)
LL misses:          51,901  (   24,812 rd  +   27,089 wr)
LL miss rate:       0.1% (    0.0%   +    0.9%  )

```

2.4.2 Processando os dados: Transposição simples X Transposição 32x32

Note que a cache de dados teve maiores valores na transposição 32x32:

- Transposição simples: 24,144,358
- Transposição 32x32: 29,162,358

Contudo as falhas na transposição 32x32 foram menores:

- Transposição simples: 10.7%
- Transposição 32x32: 6.3%

Logo, quanto menos falha na cache L1 melhor.

3 Variando o tamanho da Cache e visualizando falhas e taxa de falhas

A extensão `%%rangecachegrind` executa várias vezes com tamanhos de cache especificados pela lista `datacache=(4,8,16,32)`, em Kbytes. O usuário especifica a associatividade (**ways**) e o tamanho do linha (**line**), os gráficos são gerados de forma automática.

```

[ ]: %%rangecachegrind datacache=(1,2,4,8,16); ways=2; line=32; bargraph=(misses)

#include <stdio.h>
#include <stdlib.h>

```

```

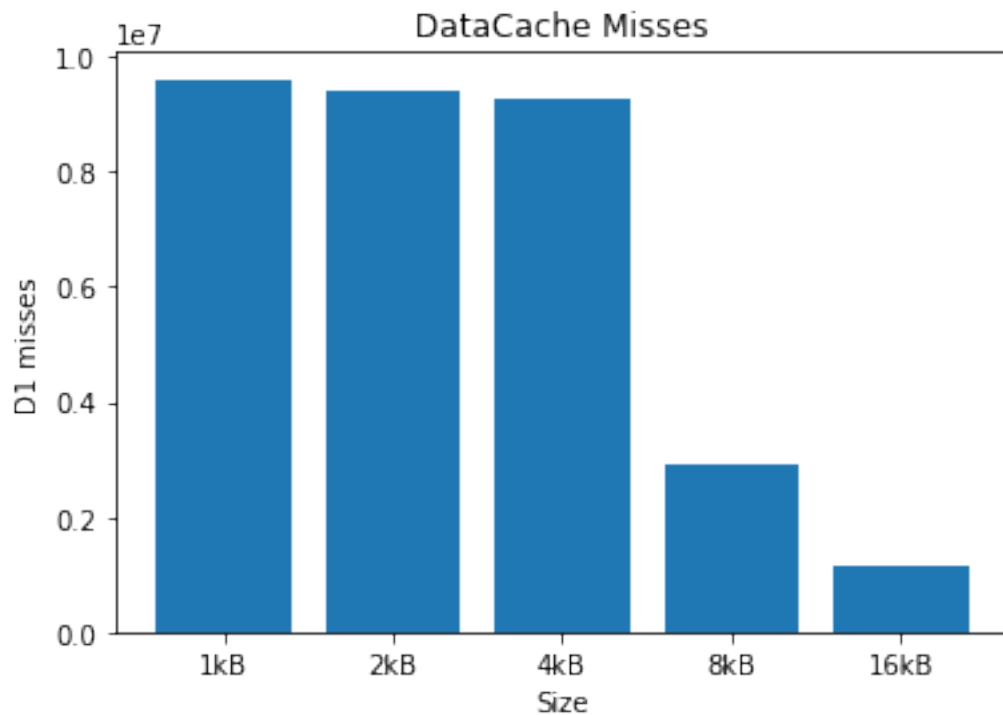
int main(int argc, char const *argv[]) {

    int n = 200;
    int a[n][n], b[n][n], c[n][n];

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            a[i][j] = i + j;
            b[i][j] = i*2 + j;
        }
    }

    int temp;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            temp = 0;
            for (int k = 0; k < n; ++k) {
                temp += a[i][k] * b[k][j];
            }
            c[i][j] = temp;
        }
    }
    return 0;
}

```



4 Tarefa

Variando os valores da cache de dados, ways e a lines (Utilize criatividade para mostrar)

```
[ ]: %%rangecachegrind datacache=(1,2,4,8,16); ways=2; line=32; bargraph=(misses)

// Experimento com algoritmo Bubblesort do grupo
// Ordena um vetor de 1000 inteiros e avalia o impacto do tamanho da cache L1
// na taxa de cache misses, variando de 1 KB a 16 KB.
#include <stdio.h>
#include <stdlib.h>

#define TAM 1000

void troca(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void bubblesort(int vetor[], int tam) {
    for (int i = 0; i < tam - 1; i++) {
        for (int j = 0; j < tam - i - 1; j++) {
            if (vetor[j] > vetor[j + 1])
                troca(&vetor[j], &vetor[j + 1]);
        }
    }
}

int main(int argc, char const *argv[]) {
    int vetor[TAM];
    // Inicializa em ordem inversa (pior caso para o Bubblesort)
    for (int i = 0; i < TAM; i++)
        vetor[i] = TAM - i;

    bubblesort(vetor, TAM); /// bubblesort

    return 0;
}
```

```
/usr/lib/gcc/x86_64-linux-gnu/7/../../../../x86_64-linux-gnu/Scrt1.o: In function
`_start':
(.text+0x20): undefined reference to `main'
collect2: error: ld returned 1 exit status
```

5 Explicação dos resultados

5.1 Análise dos Resultados - Bubblesort com Cache Variável

Os gráficos acima mostram o número de **falhas na cache de dados L1 (D1 misses)** ao executar o **Bubblesort** com 1000 elementos, variando o tamanho da cache L1 de dados entre 1 KB, 2 KB, 4 KB, 8 KB e 16 KB (com associatividade de 2 vias e linha de 32 bytes).

5.1.1 O que observamos:

- **Cache de 1 KB:** Número de misses muito alto. O vetor de 1000 inteiros ocupa 4.000 bytes (4 KB), portanto **não cabe** na cache de 1 KB. A cada iteração do loop interno, os elementos acessados foram sendo expulsos da cache antes de serem reutilizados, gerando constantes *capacity misses*.
- **Cache de 2 KB e 4 KB:** Ainda insuficiente para armazenar o vetor inteiro de 4 KB. O número de misses continua elevado, mas começa a reduzir, pois partes maiores do vetor conseguem ser mantidas em cache entre os acessos.
- **Cache de 8 KB:** A partir deste tamanho, o vetor inteiro de 4 KB já **cabe confortavelmente** na cache L1. Os misses caem drasticamente, pois após o carregamento inicial dos dados (*compulsory misses*), todo o restante das iterações ocorre dentro da cache sem precisar buscar dados na memória RAM.
- **Cache de 16 KB:** Comportamento similar ao de 8 KB. Com espaço de sobra, os misses são mínimos e referem-se apenas ao carregamento inicial do vetor.

5.1.2 Conclusão:

O experimento demonstra claramente o conceito de **localidade espacial e temporal** na hierarquia de memória. O Bubblesort, por acessar elementos adjacentes repetidamente, se beneficia muito de uma cache maior. Quando o vetor inteiro cabe na cache L1, o desempenho melhora de forma expressiva, pois os elementos não precisam ser recarregados da memória a cada passagem do algoritmo.

Isso evidencia que **o tamanho da cache L1 tem impacto direto no desempenho de algoritmos com padrões de acesso sequencial e repetitivo**, como é o caso do Bubblesort.